

■ Design Twitter/X News Feed

System Design Interview | Social Media Platform

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional

Post tweets (280 chars); Follow users; Home timeline (followed users' tweets); Trending topics; Search; Notifications

Non-Functional

350M users; 500M tweets/day; Timeline load <200ms; Celebrities: 100M+ followers; Read-heavy (100:1)

Scale

~6000 tweets/sec avg; Peak: much higher; Home timeline = hardest problem due to celebrity fan-out

■ Fan-out Architecture (Twitter's Approach)

- Hybrid fan-out: Regular users → push to followers' timeline cache on tweet; Celebrities → pull-on-read merged at query time
- Timeline cache: Redis sorted set per user {tweet_id: timestamp}; cap at 800 tweets; evict oldest
- Celebrity threshold: ~100K followers triggers pull mode; stored in user metadata; checked at fan-out time
- Read merge: for users following celebrities, fetch celebrity's recent tweets at read time and merge with pre-computed cache

■ Tweet Posting Flow

- **Write Path:** POST /tweet → Tweet Service → validate → persist to Tweet DB (MySQL/Cassandra) → publish Kafka event
- **Fan-out Worker:** Kafka consumer: fetch follower list (paginated), filter inactive users; write tweet_id to each active follower's Redis timeline
- **Inactive users:** Skip fan-out for users inactive >30 days; compute their timeline lazily when they log in
- **Media:** Images/videos: upload to S3, CDN; tweet stores media_id; async transcoding for video

■ Data Model

- tweets: tweet_id(Snowflake PK), user_id, content, media_ids, reply_to, retweet_of, created_at — MySQL (sharded by tweet_id)
- follows: follower_id, followee_id, created_at — sharded by follower_id for 'who I follow' and followee_id for 'my followers'
- timeline: Redis sorted set per user_id; members=tweet_ids; score=timestamp; ZREVRANGE returns feed
- Snowflake IDs: 41-bit timestamp + 10-bit machine + 12-bit seq → 4096 IDs/ms per machine; monotonically increasing

■ Trending Topics

- Count hashtag occurrences in sliding 1-hour window using Redis sorted set ZINCRBY hashtag count

- ZREVRANGE to get top-K trending; personalized trending = global trending filtered by user's region/language
- Storm detection: if count doubles in 5 min → trending; decay factor to prevent old trends from dominating
- Distributed counting: each server maintains local counts → periodic merge to central Redis every 30s

■ Storage Trade-offs

MySQL (Twitter's choice)	Cassandra
✓ Strong consistency	✓ Native horizontal scale
✓ Complex queries / analytics	✓ High write throughput
✓ Sharding at application layer	✓ No joins (denormalized)
✓ Read replicas for scale	✓ Eventual consistency

■ Search Architecture

- Earlybird (Twitter's real-time search index): inverted index updated within seconds of tweet posting
- Recent tweets: in-memory index for last 7 days; full index: Lucene-based for historical search
- Ranking: recency + engagement (likes, RTs, replies) + personalization (do your follows engage with this?)
- Typeahead: Trie or prefix index in Redis for autocomplete on username and hashtag search